

REST and RESTful API'leri

Web tarayıcıları diğer web sitelerinden **GET/POST** yöntemleri ile **http** ve **https** protokollerini kullanarak veri alırlar, verirler. Rest API, http protokolünü kullanır. Yani üzerinden kritik verilerin geçmesi için güvenilir değildir. Rest API genelde geliştiricilerin http üzerinden basit get/post işlemleri için kullanılır.

RestFul API'ler ise aşağıdaki özelliklere sahiptir:

- **Client-Server:** Bu iletişimde hem clien kendi tarafında verileri hem de sunucu kendine gelen verileri işler. Her ikisi de karşı taraftan bağımsız olarak verileri işleyebilir.
- **Stateless:** Bu iletişimde sunucuda yalnızca verileri işlemekte kullanılır, sunucuda veriler depolanmaz ve client'ın kendi belleğinde depolanır.
- **Cacheable:** Clientlar veri işleme performansını artırmak için kendi RAM ve işlemci cache'ini kullanır.

RestFul Implementation

Restful http üzerinden web servisini kullanır. **Dört temel özelliği** vardır.

- 1- Web servisi için <http://example.com/resources> gibi temel Uniform Resource identifier (URL)'ler kullanır.
- 2- Kullanılan data formatı web servisi tarafından desteklenir. Genelde de **JSON**, **YAML** veya **XML** kullanılır ancak bu herhangi bir geçerli html standardı da olabilir.
- 3- Kullanılan uygulamalar **http** üzerinden veri aktarımı destekler.
- 4- Kullanılan API'ler de **html uyumlu** olmalıdır.

RestFul API'leri **POST**, **GET**, **PATCH**, **PUT** ve **DELETE** gibi yaygın http yöntemlerini kullanır. Aşağıdaki tabloda ise http yöntemleri ile **RestFul karşılıkları** verilmiştir:

http Uygulaması	RESTful Karşılığı
POST	Create
GET	Read
PUT/PATCH	Update
DELETE	Delete

URI, URN ve URL Tanımları

RestFul API'ler URI kullanır. URI özel bir ağ kaynağını tarif eden adrestir. Bir URI'nin iki çeşit kullanımı vardır:

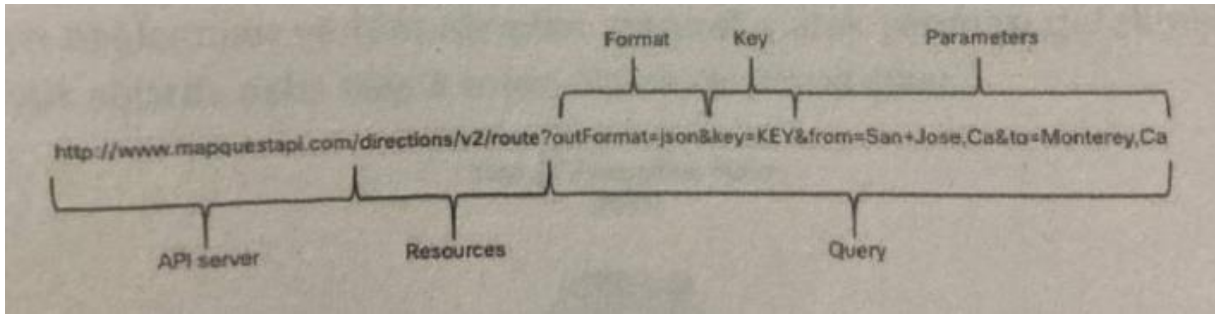
- 1- **Uniform Resource Name (URN)** – Direkt olarak kaynağın adının (web sayfası, doküman ya da resim gibi) yazılmasıyla erişilen, herhangi bir protokol kullanılmayan yöntem.
- 2- **Uniform Resource Locater (URL)** – Direkt olarak ağdaki kaynak adreslenir. http veya https kullanılarak web browser'larından erişilebilir. FTP, SFTP, SSH gibi diğer protokoller de kullanılabilir. SFTP, ftp'nin daha güvenli (secure) halidir: sftp://sftp.example.com.

Örnekte bu URI'nin parçaları <https://www.example.com/author/book.html#page155>

- Protocol/scheme : https veya ftp, sftp, mailto ve nntp gibi diğer protokoller.
- Hostname: www.example.com
- Path and file name: /author/book.html
- Fragment : #page155

RESTful İsteğin Anatomisi

RESTful Web hizmetinde, bir kaynağın URI'sine yapılan bir istek bir yanıt ortaya çıkarır. Yanıt, tipik olarak JSON biçiminde biçimlendirilmiş, ancak HTML, XML veya başka bir biçim olabilir. Şekilde MapQuest yol tarifleri API'si için URI gösterilmektedir. API talebi San Jose, California'dan Monterey, California'ya yol tarifleri içindir.



These are the different parts of the API request:

API Server – Bu, REST isteklerini yanıtlayan sunucunun URL’sidir. Bu örnekte MapQuest API sunucusudur.

Resources – İstenen API’yi belirtir. Bu örnekte MapQuest yol tarifleri API’sidir.

Query – İstemcinin API hizmetinden istediği veri biçimini ve bilgileri belirtir. Sorgular şunları içerebilir:

- **Format** – Genellikle JSON’dur, ancak YAML veya XML de olabilir. Bu örnekte JSON istenmiştir.
- **Key** – Genelde yetkilendirme için kullanılır. MapQuest, yol tarifleri API’si için bir anahtar gerektirir. Yukarıdaki URI’da, geçerli bir istek göndermek için “KEY” ögesini geçerli bir anahtarla değiştirmeniz gerekir.
- **Parameters** – Parametreler, istekle ilgili bilgi göndermek için kullanılır. Bu örnekte, sorgu parametreleri API’nın ihtiyaç duyduğu yönler hakkında bilgi içerir, böylece hangi yönlerin döndürüleceğini bilir: “from = San + Jose, Ca” ve “to = Monterey, Ca”.

Public yani herkese açık API’ler başta olmak üzere birçok RestFul API’ler bir **doğrulama** (key) gerektirir. Bu key’ler isteğin kaynağını belirlemek için kullanılır. Bu yüzden sizin API ile bir kaynağa erişmeden önce **karşı tarafın da doğrulayabileceği bir key’e sahip olmanız gerekir**. Genel olarak bir API’nin doğrulamaya (key) ihtiyaç duymasının sebepleri şunlardır:

- API’yi kullanma yetkisine sahip olan kaynağın kullandığından emin olmak için doğrulama yöntemi olarak.
- API kullanan kişi sayısını belirlemek ve sınırlandırmak için.
- Kullanıcı başına istek sayısını sınırlandırmak için.
- Kullanıcılar tarafından istenen verileri kontrol etmek ve log kayıtları için.
- API kullanan kişiler hakkında bilgi toplamak için.